PLANT ANALYSIS • SOFTWARE ENGINEERING FOR BROADCAST

CableDoc Desktop

From the Paper Patch Bay to a Directed Graph — and Back to a Facade

An exhaustive, rack-by-rack review of a cable-plant documentation tool that stopped being a spreadsheet with pretty pictures and started reasoning about its own topology

By a plant engineer turned software developer • Second edition, updated through September 2026

Anyone who has pulled an overnight master-control shift knows the scene by heart: someone in control calls to say the satellite feed just died, and the plant technician heads down to the rack with a binder of jack sheets, a marker, and the hope that whoever last touched that patch bay actually wrote the change down. Nine times out of ten, they didn't. CableDoc Desktop was born out of exactly that frustration: documenting cabling not as a dead file but as a living model you can interrogate — *"if I cut this cable, what goes dark?"* — and one that answers in milliseconds, not in half an hour of tracing wire by hand through a rack.

This is the second edition of a review first written in August 2026. The underlying codebase kept moving under our feet — a completed six-stage internal refactor, three new subsystems, and a resolution-independent coordinate model all landed in the six weeks between drafts — so rather than patch the original piece with footnotes, this edition rewrites it end to end in English and folds every subsequent delivery into the body of the text, dated where it matters.

I had access to the project's full source tree as delivered: source files, `changelog.txt` (over 500 dense entries), `schema_db.sql`, and the `plans/` folder of design documents. What follows is an engineering review, not a marketing one: which architectural decisions hold the application up, where the scar tissue of a full rewrite from an earlier VB.NET version still shows, which patterns repeat with discipline, and — because no real system is free of this — which production bugs I found documented with an honesty that few development logs bother with.

> **In one line:** CableDoc went from ``drawing the cabling'' to ``simulating the cabling,'' and then to ``teaching itself to keep its own house in order.'' That third step --- a codebase disciplined enough to refactor its own largest file down to a 145-line facade without losing a single behavior --- is the thread this second edition adds to the original story.

1. Starting point: what CableDoc is

CableDoc Desktop is a Python 3 desktop application on GTK3 (PyGObject), with SQLite as its only database — no server, no external infrastructure dependency — built to document and analyze the cabling and signal flow of a broadcast/AV plant: equipment, connectors, cables, patch bays, routing matrices, racks, rooms, and, since its most ambitious recent addition, the signal itself as a first-class entity: what content runs through a given connector, where it originates, and where it ends up.

Spec sheet (as of this edition)

- **Stack:** Python 3.10+, GTK3/PyGObject, Cairo, SQLite 3
- **Graph engine:** GraphQLite (`graphqlite.graph.Graph`), a Rust/C SQLite extension with Cypher support
- **Schema:** 47 tables, 17 SQL views (grown from 44/14 at the previous edition)
- **Python modules:** at least 34 named files across the changelog, up from 19 at first writing — the growth is almost entirely the product of the internal refactor described in §17, not of duplicated logic
- **i18n:** Spanish (base) + English + Portuguese, 300+ translated keys
- **Production data cited in the changelog:** on the order of 414 pieces of equipment, 216 patch bays, 35 GHOST-type placeholders

What makes this project interesting for a technical feature is not the feature list — long, and still growing — but that it is an uncommon case study: a very domain-specific application (TV/radio plant cabling) built by someone with exactly the field background needed to avoid the modeling mistakes that generic asset-management software makes when it tries to "guess" how a physical patch bay works. That shows in the schema, and it shows even more in the impact-analysis engine.

2. The inheritance: what survived from the VB.NET version

Before this CableDoc there was a VB.NET/WinForms version. The full rewrite to Python/GTK3 was not a simple port — it changed the rendering paradigm end to end. The earlier version generated intermediate Graphviz (`.dot`) text, shelled out to an external `dot.exe` binary to rasterize it, and in some flows even re-parsed that generated `.dot` output to extract coordinates. That is a reasonable integration pattern for its era, but it carries

an external binary dependency, a text-generation step that can drift out of sync with the model, and a parsing round-trip that is pure conceptual overhead: generate text just to read it back.

The Python version dropped Graphviz entirely. The project's own `README.md` says so plainly: Graphviz and the `dot` command are no longer needed — diagrams are drawn with Cairo, no more PDFs generated via Graphviz. All drawing — nodes, Bézier curves for patch cords, the minimap, status badges, coloring by signal or by risk — happens directly on a `Gtk.DrawingArea` with Cairo. No intermediate text, no external process, no re-parsing.

But the deeper change is not in rendering, it's in analysis. The VB.NET version, as far as I can reconstruct its scope, had no real impact-simulation engine; the graph, where it existed, was mostly for drawing. The Python version introduced `graph_impact.py`, which builds the full signal graph in memory using **GraphQLite**, a SQLite extension written in Rust and C with Cypher query support. That is an uncommon architectural choice, and in my judgment the right one: instead of hand-rolling a BFS/Dijkstra over Python dictionaries — which works, but scales poorly and is easy to get subtly wrong — the project leans on a real embedded graph engine with its own query language, no separate server required.

The one piece that crossed over intact

Of the entire data-access layer of the prior version, exactly one piece survived the full rewrite unchanged in substance: the SQL view `CONEXIONES_AMBOS_EXTREMOS`, a self-join over the base view `CONEXIONES` that resolves both ends of a cable (equipment, type, connector) into a single row. Eleven years of UI code — CRUD forms, exports, the graph engine itself via `_leer_bd()` — still query it by column position. A single piece of logic surviving a full rewrite of framework, rendering language, and analysis engine says something about how well that query was designed in the first place.

3. Anatomy of the code: where complexity lives

An honest review can't skip the size distribution of the files, because that's where you can see — without reading a single line — where maintenance risk is concentrated. At first writing, `pantallas_avanzadas.py` was the elephant in the room at 9,830 lines. It no longer is: §17 covers the six-stage effort that took it down to a 145-line pure facade. What follows is the shape of the codebase *today*.

Where the lines live now

- `cabledoc.py` — main window, menus, every CRUD dialog; still the single largest file (grew slightly, then shrank again once the legacy node editor was removed in Entrega 6)
- `modelo.py` — the entire data-access layer, now past 5,000 lines after absorbing splice tracking, percentage coordinates and the ghost quick-add helpers
- `diagrama_conexiones_ui.py` — 1,694 lines; the connection-diagram window itself plus its remaining satellite dialogs, composed from **fifteen** mixins spread across as many files
- `patcheras_ui.py` — 1,519 lines; the global patch-bay view, extracted whole in Entrega 3
- `cypher_console.py` — 1,864 lines (interactive Cypher console)
- `graph_impact.py` — 1,369 lines (the signal graph engine)

The mixin-by-inheritance pattern that already organized the diagram window at first writing — `ImpactoMixin`, `RiesgoDiagramaMixin`, `SenalDiagramaMixin`, `EscenarioMixin`, `DiagnosticoMixin` — has since been extended to the diagram's own internals: graph construction, Cairo drawing, mouse/keyboard interaction, wire editing, auto-layout, search, export, and internal patch-bay routing each now live in their own mixin file. The complexity of each subsystem stays isolated in a 200-to-1,700-line module of its own; what used to accumulate in one host class is now, quite literally, composed from fifteen well-named pieces. It's a reasonable composition pattern for GTK3, which offers no declarative plugin/slot system the way modern web frameworks do — and, as §17 shows, the project eventually decided the host file itself needed to shrink too, not just gain more mixins forever.

3.1 The data model: catalog vs. instance, still strict

The SQLite schema now has 47 tables and 17 views. One design detail that keeps showing up in the changelog, and is worth calling out because it's easy to overlook in smaller projects: the strict separation between *catalog* tables (reusable templates: `equipo_catalogo`, `conector_catalogo`, `slot_catalogo`, `frame_catalogo`) and *real instance* tables (`equipo`, `conector`, `slot`, `frame`). When the AND/OR logic-rule engine was added for equipment gates (modeling a DSK or any multi-input conditional combiner), the project didn't mix rule definition with rule instance: it created `regla_logica_catalogo` / `_miembro` / `_salida` as tables separate from `regla_logica` / `_miembro` / `_salida`, deliberately mirroring the same pattern equipment and connectors already used. That same discipline was applied again a month later to the two biggest additions in this edition — splice tracking and electrical-format fields — and it's the kind of discipline that's easy to drop under deadline pressure and, here, wasn't.

4. The core of the system: the signal-impact engine

If I had to point to a single piece of code as the most valuable in the whole project, it would still be `graph_impact.py`

and its `GraphImpactAnalyzer` class. Not for its size (1,369 lines, modest next to the UI files) but for what it solves: turning a physical cabling topology — with all its ambiguity of patch bays in bypass, routing matrices that may or may not be configured, and logic gates — into a queryable directed graph, and answering three real operational questions over it:

- `simular_desconexion(cable_id)` — what loses signal if I cut this cable?
- `simular_falla_equipo(equipo_id)` — what loses signal if this piece of equipment fails completely?
- `simular_escenario(…)` — the generalization of both to an arbitrary combination of changes, evaluated in one pass, meant for planning an emergency reroute before touching a single real cable.

What makes the model sophisticated isn't the reachability algorithm itself — fundamentally a nodes-reachable-from-a-seed-set problem over a graph with excluded edges — but the fidelity with which it translates real physical behavior into graph structure. Two bugs from the project's own log illustrate this well.

Real bug #1 --- the patch bay that hid impact

A patch-bay module was originally modeled as a single "passes everything" node. Its four physical ports (rear in, rear out, front tap, front insert) shared one logical node. Consequence: cutting the cable feeding the input port didn't isolate what came out the other three ports if *any* other cable still touched that equipment through any connector — impact analysis underestimated real downstream damage. The fix modeled each of the four ports as its own node (`PP:<connector_id>`) with internal edges matching real physical bypass: with nothing patched in front, input→output flows straight through; if the tap and/or insert is used, that bypass breaks and signal follows the front instead. Validated against a real case from the database: before the fix, cutting a specific cable returned `hay_impacto=False`; after, it correctly detects 45 pieces of equipment and 55 cables impacted downstream.

Real bug #2 --- the rule that lost to stale data

The second bug is subtler and, to me, more interesting from an engineering standpoint: a piece of equipment with its own logic rule (AND/OR, e.g. a DSK with multiple conditional inputs) could *also* have leftover rows in the matrix-routing table for that same equipment — stale data from a configuration loaded before the rule was defined, or from a later role change. Both paths competed for the same input connector, and matrix routing silently "won," leaving the rule's gate node with a member that, in practice, no cable fed. The production symptom: impact analysis detected *no* downstream change from any cut near that equipment, because it already showed as signal-less — a perfectly silent false negative. The fix gave explicit priority to the equipment's own rule over any residual matrix row, and confirmed only 1 piece of equipment in the entire real database had this conflict. Worth noting the diagnostic method: not a code review, but a user report ("there's no way this has zero impact") followed by a data-driven investigation to the exact root cause.

Both bugs share something that speaks well of the process: neither was closed with a plain "fixed, it works now." Each has its correction validated against a copy of the real database — never the original file — with concrete before/after counts, documented in the changelog with the same level of detail an SRE team would give a real incident post-mortem.

4.1 Risk as an explicit function, not a gut feeling

Built on the same graph engine, `risk_engine.py` computes a Failure Risk Index (IRF) per piece of equipment with a deliberately simple, auditable formula: `Risk = max(Probability, P_MIN) × Impact / 100`. Probability blends equipment age, usage condition and logged problem history; Impact directly reuses `GraphImpactAnalyzer.simular_falla_equipo` to compute what fraction of the plant loses signal if that equipment fails outright. A product detail I find well thought out: if a hand-marked set of "critical equipment" exists — say the full on-air chain, marked via multi-select right on the diagram — Impact is measured only against that set, not against the whole plant. The practical difference is large: without that adjustment, a master switcher and a secondary control-room monitor weigh the same in the impact calculation if both affect a similar relative fraction of total equipment. With the critical set active, risk prioritizes what actually matters, and if the critical-set table is empty the system falls back explicitly to the whole-plant criterion, documented as expected behavior rather than an unhandled edge case.

5. Testing without a traditional safety net

It's worth a paragraph on how this project is actually tested, because it's atypical and the changelog is transparent about the limits. There's no traditional `pytest` suite running UI tests in CI — unsurprising for a GTK3 project of this size, where automating real mouse/widget interaction usually costs more than it's worth for a single-developer team. Instead, UI validation runs real GTK under Xvfb (a headless X server), instantiating real windows, firing real signals via `GLib.idle_add()`, and checking for exceptions and unexpected behavior — against a copy of the real database, never the original.

That discipline hardened noticeably during the refactor described in §17: an Entrega-2 bug that only manifested when a drawing callback actually executed — invisible to both `ast.parse` and a successful module import — established `pyflakes` as a mandatory validation step from that point forward, not an optional extra. It's a good case study in "know your tools' blind spots": GTK silently swallows exceptions raised inside a `draw` callback, turning a `NameError` into a blank black surface with zero traceback, so neither syntax checking nor a

clean import will ever catch it — only actually exercising the callback, or a linter that resolves every name statically, will.

6. The diagnostic assistant: bisection applied to a plant problem

One of the more recent additions — and, to my mind, one of the most elegant from an algorithm-design-applied-to-a-concrete-domain-problem standpoint — is `diagnostico_falla.py`. The idea is simple to state and not trivial to implement well: given a symptom ("no signal here"), instead of a technician following the cable physically leg by leg, the system reconstructs the full chain of connectors between the symptom point and the source — reusing the same inverted graph that `senal_visual.py` builds — and runs **real bisection** over that chain: it asks about a midpoint, discards the corresponding half based on the answer, and repeats. It's binary search applied to physical cabling topology, with the added wrinkle that "the middle" isn't an array index but a physically accessible test point (`conector.es_punto_test`), auto-populated for patch-bay front jacks and hand-editable for any other point.

Every cut that fails to reconstruct the full chain is classified into one of six explicit categories — `FUENTE`, `MATRIZ_SIN_RUTEO`, `PATCHERA_INCOMPLETA`, `BIFURCACION`, `SIN_ORIGEN`, `BUCLE` — instead of a generic "could not determine." That classification isn't cosmetic: it tells the technician *what kind* of problem they're looking at, which is different, actionable information in each case.

7. Internationalization: knowing when to automate

CableDoc was born in Spanish and internationalized to English and Portuguese without touching the data model or rewriting the UI. The mechanism is deliberately simple: a 300-plus-key `i18n.py` dictionary (key = Spanish text, value = per-language translations) accessed through a gettext-style `_()` function. What makes the process interesting, more than the result, is the application pipeline: rather than hand-translating hundreds of scattered string literals across 8,000-plus lines of GTK dialogs, the project built `auto_wrap.py`, an AST-based script that automatically wraps relevant string literals in `_()` calls, operating by byte offset to avoid the risk of a naive text replacement corrupting similar strings in different contexts.

That automated approach covers the general case but — as with any AST transform over real UI code — leaves out cases that require judgment: interpolated f-strings, literals inside tuples, dynamically built radio-button text, and constants evaluated at import time (the documented case is `SNIPPETS` in `cypher_console.py`, a module-level constant that had to become a function, `_construir_snippets()`, so translation resolves at the right moment instead of at module-load time, before the language is even configured). This is a lesson worth generalizing to any refactor-automation pipeline: automate the mechanical 90% and treat the remaining 10% as a known special-case list, rather than forcing the script to cover 100% at the cost of ever more brittle rules. In late August, a full translation pass swept the roughly ten `*_ui.py` files that had shipped features without translations, and in the same pass caught a genuinely blocking bug: nine of those files used `_()` without importing `i18n` at all, which meant a real `NameError` the first time a user opened, say, the Impact menu.

8. Recurring patterns (for the better)

A handful of conventions show up again and again, delivery after delivery, and — for anyone who has maintained a long-lived project — are the difference between a codebase you can keep extending with confidence and one that starts to feel scary to touch.

Conventions sustained over time

- **Idempotent migrations with an explicit sentinel.** Every `asegurar_tablas_*()` / `asegurar_columnas_*()` in `modelo.py` checks whether the column/table already exists before populating it, and *data* migrations (not schema ones) use a dedicated control table as an "already run" sentinel when there's no new column to key off of.
- **Never silently overwrite a local value on import/export.** The JSON catalog importer never overwrites a value that already differs at the destination — it produces an explicit conflict list, resolved field by field in a dedicated dialog, with "keep local" as the default.
- **Validate against a copy of the real database, never the original file.** The single most repeated phrase in the changelog, almost ritual at this point, and the correct practice.
- **Batch instead of N+1.** When a one-query-per-node pattern shows up in a render loop, it gets replaced with a batch query — one documented, real measurement drops diagram load time from roughly 2.2 seconds to roughly 8 milliseconds.

9. Friction zones and visible technical debt

No serious review is praise alone. A few concrete points I'd flag before a long sprint of new features:

Two files still concentrate a lot of surface. `cabledoc.py` and `diagrama_conexiones_ui.py` together still hold a disproportionate share of the project's Python. The mixin pattern keeps *new* logic isolated, but the host classes remain mandatory integration points that keep growing. Not urgent — GTK3 doesn't leave many better alternatives short of a custom plugin framework — but it's the kind of file that, eventually, costs more to open in an editor than to reason about.

Positional indexing over a wide view. The documented decision *not* to add new columns to `CONEXIONES_AMBOS_EXTREMOS`, to avoid disturbing its column order because several call sites read it by numeric

position, is pragmatic short-term and a long-term fragility source. Any future change to that view requires auditing every consumer by position rather than letting the language itself (named columns, or an explicit dataclass mapping) guarantee correctness by design.

Accumulated compatibility fallbacks. Several "free text to control column" migrations — connector direction, patch-bay role, port function — shipped with a compatibility `OR` against the old criterion instead of a clean cut. The right call at the time, but worth noting the project has also been actively *removing* several of these once confirmed unnecessary — the correct discipline of introducing a fallback and scheduling its removal, rather than leaving it indefinitely "just in case."

10. Scenario mode: plan before you touch a cable

`escenario_engine.py` (341 lines, deliberately GTK-free) orchestrates the model and graph analyzer to let a user assemble, in memory, a combination of changes — failed equipment, cut cables, proposed virtual reconnections — and evaluate their combined effect with `simular_escenario()` before writing a single row to the real database. The design detail I value most here is the explicit split between what can be *simulated* and what can be *applied*: equipment failures exist for simulation only, so `aplicar_a_infraestructura()` ignores them on purpose and only materializes cable reconnections, inside a single explicit transaction — and before applying anything, `resumen_aplicar()` builds, without touching the database, exactly what will be deleted and inserted, for a confirmation dialog. That's the difference between a tool that "does things" and one that lets you verify the plan before executing it, which in a live on-air emergency is a real operational safeguard, not a UX nicety.

11. The inverted graph: one trick, three problems

A cross-cutting design decision worth calling out on its own: `senal_propagation.py`, `senal_visual.py`, and, in bisection form, `diagnostico_falla.py` all solve superficially different questions — *what signal name runs through this cable?*, *what reference image corresponds to this output connector right now?*, and *where exactly is the break?* — but underneath they're the same problem: given a connector, walk backward to find who feeds it, crossing the same passthrough equipment types (DISTRIBUIDOR, ENRUTADOR, MODULO PATCHERA) with the same bypass semantics. Instead of tripling that logic, `senal_visual.py` explicitly builds an inverted graph `REV[connector] = the connector that feeds it`, describing itself in its own docstring as reusing "the same inverted-graph trick the rest of the project already uses." It's a clean example of how modeling a problem well once (who feeds whom?) lets you answer several apparently different business questions with the same data structure.

12. Signal risk: from plan to code

Between the first and second editions of the review the biggest single feature landed: `signal_risk.py` and its `SignalRiskAnalyzer`, computing, for every cable, three deliberately independent axes — never merged into a single score, so the UI can show the specific cause instead of an opaque number.

Three axes of signal risk

- **Attenuation** — an analog run too long for its cable type (per-run flag, no cumulative chain math in this first version).
- **Bandwidth bottleneck** — a cable with markedly less bandwidth than its neighbors sharing the same equipment (a classic case: an old patch cord wedged between modern gear).
- **Format mismatch** — incompatibility between the two ends of a cable: electrical (conductor mismatch risking a short), balance, or channel.

The interesting schema detail here is one the original plan hadn't anticipated: `conector` had no column describing its real electrical format — `tipo_conector` is a catalog of physical shape, not signal format — so `conector.id_tipo_ficha` was added as a new foreign key into a new electrical-jack catalog (`tipo_ficha`: conductor count, default balance and channel mode, bandwidth). The engine was validated against a synthetic scenario built to exercise every axis, and against the 496 real cables in the database with zero false positives.

A post-delivery design bug, fixed one day later and documented with the same honesty as everything else: the first attempt modeled a cable's electrical jack as a single column on `cable` — one value for the whole cable. A real user-supplied case disproved it: a cable can have an XLR3-male jack on one end and a TRS jack on the other, perfectly normal on a field adapter, which one field per cable can't represent. The fix — `conexion.id_tipo_ficha`, nullable, one per connection row — is the same idea the project already used elsewhere: when two ends of the same entity can differ, the correct datum doesn't live on the shared entity but on each connection point, which already knows unambiguously which connector it corresponds to. The changelog explicitly records having considered and rejected the obvious alternative — `cable.id_tipo_ficha_extremo_a/b` — for a subtle, well-identified reason: there's no stable ordinal that distinguishes "end A" from "end B," because `CONEXIONES_AMBOS_EXTREMOS` itself builds that label with a self-join and no `ORDER BY`, so anything depending on that distinction would be non-deterministic by construction.

13. Incident log, workmanship, and analog risk

The second major delivery of that window is a full new subsystem: an incident log (`incidente`, `zona_sospechosa` as a reusable equipment group, linked to equipment/cables/zones)

combined with a per-connector and per-cable "correct workmanship" flag, aggregated in `riesgo_analogico.py` into a per-equipment/cable/zone *hot-zone* score, with linear decay by age inside a configurable window.

The richest bug in this delivery, again motivated by a real photo a user brought in, repeats almost verbatim the granularity pattern that had just resolved the electrical mismatch a few hours earlier: the original "correct workmanship" system lived at the whole-`cable` level and at the equipment-`conector` level — neither could represent "this specific end of this specific cable is badly terminated," exactly the case the client brought in: a cable with an XLR3 jack on one side and TRS on the other, with a technician having crossed conductors on only one of the two jacks, invisible on inspection. The fix, consistent with the electrical-mismatch fix a day earlier, extended the same pattern one level down: `conexion.es_armado_correcto` / `detalle_armado`, per connection row, leaving the whole-cable flag intact as an independent quick shortcut. The changelog documents first considering and discarding a new table (`problema_ficha_cable`) on noticing the overlap with the existing workmanship system — avoiding a parallel table when the right move was extending an existing pattern is, again, the same DRY discipline seen throughout this codebase.

By the end of August this subsystem had a real front end too: `ZonasSospechosasListado` in `bitacora_ui.py` gives suspicious zones their own catalog screen (create, edit, delete, jump straight to their incident log) instead of only being reachable "in passing" from an incident's zone picker — closing a real gap a user hit in the field: a zone-only incident (no equipment, no cable) saved correctly to the database but had no UI path back to it.

14. Splices become first-class: extension_cable

A gap the model simply didn't cover before this delivery: two loose cable ends spliced jack-to-jack, with no equipment or barrel connector in between — common in the analog chain behind a wall plate or inside a rack. Before this feature, no intermediate leg like that had anywhere to be recorded in CableDoc; every cable end was required to terminate at a real equipment connector. An *Extension* (`extension_cable`) is exactly that: the point where two cables splice directly into each other, explicitly distinct in the project's vocabulary from an *Empalme* (barrel/coupler), which already meant something else in the schema.

The feature shipped across four phases: schema (a new table, with `conexion.id_conector` confirmed already nullable so no `ALTER TABLE` was even needed), CRUD and dialogs in a new `extension_cable_ui.py`, propagation/impact integration, and finally a risk-scoring weight for a badly-made splice — which, marked incorrect, contributes its own weight to the analog-risk score *and* heats up both spliced cables, so the existing overlays flag them even before the diagram gained its own visual node for splices.

Real example: three legs

The case that motivated the feature: `DDA 04 ESTUDIO` (OUT 3) → `MONITOR DIRECTOR CAMARA` (a Roland MA-12C), across three cable legs joined by two splices. A "View full chain" feature (`resolver_cadena_extension()`) walks the splice chain in both directions from any cable in it, all the way to a real equipment connector on each side — or to a loose end / a break, if the chain is still incomplete — rendering something like:

```
DDA 04 ESTUDIO -- OUT 3
    | cable MONITOR C11 DIR CAMARA
Splice #3 (behind computer desk) -- unverified
    | cable ...ALARGUE...ADAPTADOR
Splice #4 (near director's monitor) -- unverified
    | cable ...ADAPTADOR XLR3 A PLUG TS
MONITOR DIRECTOR CAMARA -- INPUT JACK 3
```

That view opens automatically the moment a splice is created or edited — direct usability feedback added after a real complaint ("I don't know what I'm doing at each stage of building the extensions"), rather than making the user hunt for a separate "view chain" button after the fact. Both the impact engine and signal propagation now cross splices transparently when walking a chain; the diagram itself renders each splice as a small diamond marker between the two real endpoints, though that visual layer doesn't yet feed risk, scenario, search, or export — an explicitly documented limitation, not an oversight.

15. Ghosts get a fast path

Documenting a confirmed-dead cable end used to mean: write the name `EXTREMO A/B DESCONECTADO <code>` on a separate notepad, open "New equipment," copy that name in, pick type GHOST, fill in (empty) Brand/Model/Inventory/Serial fields anyway just to get through the form, choose IN or OUT by hand, and only then go load a photo and a location. A single button now does it in one step: **" Mark end disconnected"** on a cable's own detail dialog. With zero ends already loaded it asks a one-shot question (end A, which becomes OUT, or end B, which becomes IN); with one end already loaded — real or ghost — it infers the opposite side on its own, no prompt needed; with two ends loaded, the button is simply disabled. Underneath, it creates the GHOST equipment, its single connector, and the connection in one step, deliberately skipping Brand/Model/Inventory/Serial and chaining straight into the existing equipment and connector dialogs for an optional photo and plan location, exactly the fields a technician might not have handy in the moment.

A companion feature closes the loop a step later: when reconnecting a cable over an existing connection in the diagram's drag-and-drop wiring flow, if the *other* end of the cable being displaced turns out to be a GHOST with no other connections left, CableDoc now detects that on its own and offers to delete the orphaned ghost right there, instead of it lingering in the database until someone remembers to clean it up by hand. It's scoped narrowly and honestly: it only fires on the specific reconnect-and-replace flow, and it only ever looks at the *displaced* cable's

other end, never the destination connector (which, by construction, always receives the new connection and so can never be orphaned by this operation).

16. Removal simulation, live signal state, and lineage

Three smaller but operationally valuable additions round out this window, all sharing the same graph engine except the last, which is pure documentation.

Simulate removal puts a lightweight version of impact analysis directly on the detail dialog of a Cable, Equipment, Rack, or Connection — so a technician editing one record doesn't need to open the full diagram just to sanity-check a change. A Rack simulation cuts power to everything physically mounted in it, evaluated as one combined event (important for an AND rule spanning two pieces of equipment in the same rack); a Connection simulation is treated as pulling the whole cable, because the signal graph doesn't model a loose single end separately.

Signal state (down/up) is not a button but an overlay: while any of Impact, Risk's "simulate failure," or Scenario mode is active and "color by signal" is on, connectors that lose a manually-loaded signal name show it struck through instead of silently displaying it as if nothing happened, with the legend gaining an explicit " down (active analysis)" entry. It's purely read-only — a new `senal_estado.py` cross-references whichever engine's impacted-equipment set is currently cached against `senal_en_conector` and reports which connector had which name; it recomputes nothing and writes nothing.

Signal lineage documents which other named signal(s) a given signal derives from — useful when a DSK or standards converter changes a signal's name in a way `senal_propagation.py` deliberately can't infer on its own. It is explicitly, by design, pure documentation: it does not feed impact analysis, propagation, or any overlay in the diagram. An auto-suggestion algorithm looks only at manually-loaded *output* connectors for the child signal and proposes whatever signals are loaded on that equipment's inputs as candidate parents — deliberately ignoring input-loaded occurrences of the child, since a signal arriving on an input doesn't mean the equipment produces it. A cycle check runs before saving each parent link and rejects only the offending row, not the whole save.

17. The great facade: the refactor completes

The single largest engineering story of this window is process, not feature: the six-stage plan to break up `pantallas_avanzadas.py` — 9,830 lines at first writing, 11,032 at the refactor's actual baseline once later additions were counted — finished completely. The file is now 145 lines: a pure facade with zero classes or functions of its own, re-exporting names so that neither `cabledoc.py` nor `diagrama_personalizado.py` needed a single import changed.

The refactor in numbers

- **Entrega 0:** `pantallas_comunes.py` extracted — shared i18n bootstrap, icon caching, `_ImagenZoom`, the color palette, stateless Cairo helpers.
- **Entregas 1–2:** `arbol_conexiones_ui.py`, `frame_slots_ui.py`, `imagen_conectores_ui.py`, `rack_ui.py` split out. 11,032 → 8,863 lines.
- **Entrega 3:** `patcheras_ui.py` (1,519 lines) extracted whole. → 7,380 lines.
- **Entrega 4:** the two catalog/instance editor pairs deduplicated into shared base classes across two new files. → 5,550 lines.
- **Entrega 5:** 70 methods (~2,840 lines) pulled out of `DiagramaConexiones` into 8 new mixins. → 2,712 lines.
- **Entrega 6:** `diagrama_conexiones_ui.py` (1,694 lines) takes the last classes whole; the legacy custom node editor, `EditorConexiones` (~900 lines), is deleted outright rather than ported, since its menu entry had been disabled since quick-wiring started reusing the diagram canvas. → **145 lines**.

Two bugs from this run are worth dwelling on, because each taught the project something that changed how the *next* stage was validated.

Entrega 2 --- the callback that GTK ate silently

Extracting `VistaRack` pulled two small Cairo text-drawing helpers (`_tc`, `_abrev`) along with it into `rack_ui.py` — except they weren't exclusive to `VistaRack`: `PatcherasVista` and `DiagramaConexiones`, still living back in `pantallas_avanzadas.py`, called them too, and were left calling undefined names. Because the failure happens inside a GTK `draw` callback, the resulting `NameError` is swallowed silently by GTK/Cairo: no traceback, just a blank black surface or missing nodes. Neither `ast.parse` nor a clean import under Xvfb had caught it, because the code path only executes when the callback actually runs. `pyflakes` became mandatory validation from this point on — it caught this bug plus two more latent ones from the previous stage that had simply never been exercised yet.

Entrega 5 --- a decorator on the wrong side of a boundary

The extraction script computed each method's line range as "up to the line before the next `def`." Any decorator line (`@staticmethod`) sitting immediately before a `def` was, by that rule, attributed to the *previous* method in the source file rather than to the one it actually decorated. Exactly one such case existed in the whole `DiagramaConexiones` block: the `@staticmethod` meant for `_seg_intersect`

ended up glued onto `_draw_conexion_interna` instead, which surfaced in production as `TypeError: _draw_conexion_interna() missing 1 required positional argument: 'cr'` — reported by the developer with a real screenshot and traceback. The fix was one line moved to the right place; the verification was the interesting part: a byte-for-byte comparison of all 84 methods of `DiagramaConexiones` against their pre-refactor bodies, confirming the other 81 were untouched.

Why this matters: a project that runs its own refactor as a numbered, changelog-tracked plan --- with each stage's exact before/after line count, each bug traced to a root cause and verified with a mechanical diff rather than a shrug, and a final stage that chose to *delete* nearly a thousand lines of legacy code rather than carry it forward out of habit --- is treating internal architecture with the same rigor it applies to signal-impact correctness. That discipline, more than any individual line count, is what makes it credible that the next six weeks of feature work will land on solid ground.

18. Quick wiring: the diagram becomes an editor

A smaller but very visible change: "Alta rápida de conexiones" (Quick wiring), previously its own bespoke node editor (`EditorConexiones`), was rebuilt on top of the connection diagram itself (`DiagramaConexiones(iniciar_vacio=True)`) — it opens empty rather than pre-populated, but gets every diagram tool for free: zoom, search, export, impact, risk, signal overlays, scenario. A side panel lets a technician search and drag equipment onto the blank canvas (with an optional "bring connected equipment" checkbox, on by default, that auto-expands a newly-added piece of equipment's existing neighbors the first time it's placed); dragging from one connector to another creates a cable between them, reusing the same quick-cable popup the rest of the app already had.

The nicest piece of integration here is that this canvas now understands the "incomplete connections" overlay (documented cable ends still missing their other side, shown as a dotted orange leg with a cone marker) well enough to skip its own popup: dragging from a connector that already carries a one-sided cable straight onto a new connector reuses that exact cable instead of prompting to search or create one — unless *both* ends of the drag turn out to have their own incomplete cable, in which case the tool correctly refuses to guess which one should win and just says so in the status bar. A companion "auto-arrange, no overlap" button (session-only, deliberately not persisted to the database) and a Ctrl+E shortcut that expands the neighbors of an entire multi-selection at once, deduplicating shared neighbors between selected nodes, round out a screen that went from a separate, narrower editor to simply another mode of the main diagram.

19. Vector graphics and resolution-independent coordinates

The most recent delivery in this window, and a quietly significant one: CableDoc now accepts SVG images for equipment, connectors and frames alongside the usual JPG/PNG/GIF/BMP/WebP, and — more consequentially — every stored connector/slot position and rectangle (X, Y, width, height) now persists as a *percentage* of the underlying image's width/height rather than as raw pixels, with **no schema change at all**: the same columns simply now hold a different unit. The immediate practical payoff: replacing an equipment's image with one of a different resolution used to leave every positioned connector visually wrong, because pixel coordinates no longer matched; percentages survive a resolution change unchanged.

Image dimensions themselves aren't stored anywhere — there's no column for it on `imagen` — so they're read live off the actual file (GdkPixbuf for raster formats, native Rsvg for SVG without ever rasterizing it) with an in-memory cache keyed on path and modification time. The conversion is entirely hidden inside `modelo.py`: every other layer of the app — Cairo drawing, `_ImagenZoom`, every screen that positions a connector — keeps working in pixels exactly as before; the model layer converts on the way in and out. A one-time, explicitly destructive migration (`migrar_coordenadas_a_porcentaje()`, idempotent via a marker table) converted the existing production database: 2,751 rows migrated, 20 controlled errors (connectors with no image assigned yet), and a documented data finding worth a manual pass later — a few dozen points came out above 100% (up to roughly 190%) because they were already positioned outside the image's border before the migration ran; the migration deliberately does not clamp them back on-frame, since that would silently discard real position data.

The SVG rendering itself is worth a technical footnote for anyone who has to make the same call: the project explicitly chose the modern `render_document` API over the deprecated `render_cairo` (removed from librsvg 2.58), and verified the choice with a test that fails on any `DeprecationWarning` rather than trusting a changelog entry from the library. A lightweight `_ImagenSVG` wrapper duck-types `get_width()`/`get_height()` to match `GdkPixbuf.Pixbuf`, so the half-dozen call sites across four different UI files that read `.pixbuf` directly kept working unmodified — one more instance of the project choosing an interface shape specifically so a downstream refactor touches nothing.

20. The pocket edition: Kivy on Pydroid 3

A parallel, ambitious effort: a full migration of CableDoc to **Kivy**, aimed specifically at running on an Android phone through **Pydroid 3**, without a traditional APK build step. As of this edition it documents six complete, real-data-validated phases, with a seventh explicitly marked optional and low priority.

CableDoc Kivy --- surveyed state

- **Scope:** 100% of CRUD modules and the 3 main graphical views (rack, frame/slots, patch bays) migrated and tested
- **Code:** ~15,200 lines across 20 Python files
- **Reused unmodified:** `modelo.py` (the entire data layer) and `i18n.py`
- **Pending (phase 7, low priority):** the full interactive impact diagram and bulk coordinate editors

What jumps out, and speaks to the quality of the original split between data and presentation layers, is that `modelo.py` — nearly 5,000 lines of SQLite access underpinning the entire desktop app — is reused in the Kivy version **without a single change**. The same `db.db`, the same schema, the same parameterized queries: only the presentation layer changed end to end. It's the practical proof, beyond any theoretical argument about separation of concerns, that keeping that boundary clean over years of desktop development paid off: when it came time to port to a completely different UI framework on a completely different operating system, the larger, more critical half of the code — the half that guarantees data-model integrity — didn't need to be touched.

The deepest architectural change between the two UIs is not cosmetic, it's about the UI concurrency model: `Gtk.Dialog.run()` in GTK3 is blocking, letting synchronous, linear code drive a dialog's result. Kivy has no blocking equivalent — every `Popup` opens with `.open()`, which returns immediately, and the user's answer arrives later through a callback. The migration applied that transformation — linear flow to callback flow — consistently across every dialog in the project, a control-flow paradigm shift that runs through every one of the twenty files, not a one-off patch in a couple of screens.

21. Sync without a server: Google Drive via `rclone`

A structural limitation of choosing SQLite as the sole database — excellent for deployment simplicity, no server process to run — is that the file lives on one local disk. The chosen fix is deliberately low-maintenance: `rclone`, treating a cloud storage provider like just another remote filesystem, syncing the `database/` folder against Google Drive. On the mobile side, sync runs through **RoundSync**, an Android app that wraps `rclone` with a GUI and task scheduler, pointed at the same remote. The practical result is one circuit — desktop ↔ Google Drive ↔ phone — with `rclone` as the common denominator on both ends, and the same single-active-writer discipline applying on both sides: sync before you start working on whichever device you're about to use, and sync again when you finish.

22. A shift on call, reconstructed

To keep the rest of this analysis from staying purely abstract, it's worth reconstructing — chaining together real, shipped functionality — what using CableDoc looks like at the exact moment it was built for: not a calm desktop planning a migration, but mid-shift, with something off the air.

It's three in the morning and control calls: a satellite channel just lost signal. The on-call technician opens the connection diagram, switches on " Diagnose failure" from the Scenario submenu, and clicks the output connector on the monitor where the cut was noticed. The assistant reconstructs the full chain back to the source — crossing, without the technician needing to know it, two patch bays in full bypass and a matrix with routing configured — and starts the bisection: it asks about a test point halfway along, physically reachable at a patch-bay front jack. The technician checks it with a handheld tester, answers "No," and the correct half of the chain is discarded in one step. Three questions later, the system isolates the exact leg: a cable between a distribution amp and the first patch bay. It isn't magic — it's a well-instrumented binary search over a well-modeled graph — but for someone who would otherwise be tracing that same cable by hand through the rack with a flashlight, the difference between three questions and forty minutes of manual tracing is the difference between fixing the problem before or after the affected air block ends.

23. Quick reference: the analysis-engine module map

Core analysis modules

- `graph_impact.py` — builds the full directed graph, answers impact of cuts, equipment failures and combined scenarios.
- `risk_engine.py` — per-equipment Failure Risk Index, combining probability with impact measured over the graph above.
- `signal_risk.py` — catalog-driven prediction of attenuation, bandwidth bottleneck and format mismatch, per cable.
- `riesgo_analogico.py` — incident-log and workmanship-driven hot-zone scoring, with linear age decay.
- `senal_propagation.py` — signal-name propagation engine (Bellman-Ford, no GraphQLite).
- `senal_visual.py` — inverted graph resolving which reference image belongs to a connector right now.
- `senal_estado.py` — read-only overlay marking which loaded signal names are currently down.
- `diagnostico_falla.py` — real bisection over the chain between symptom and source.
- `escenario_engine.py` — orchestrates simulation and real application of combined changes, GTK-free.
- `cypher_console.py` — direct access to the graph engine's own query language.

None of these ten modules reimplements its own graph traversal from scratch. All of them read the same notion of "who feeds whom," built once from control columns (`direccion`, `rol_senal`, `id_funcion_patchera`) that a multi-phase de-hardcoding effort left as the single source of truth. It's the most concrete proof that migration work — tedious, with no new visible feature the day it shipped — is what made every operational tool described in this review possible to build on a shared foundation.

24. Domain glossary for a non-broadcast reader

Plant vocabulary

- **Patch bay:** a panel of connectors where each piece of equipment's cable terminates at a rear jack, optionally letting you "intercept" the signal from a front jack without unplugging the original cable. With nothing patched in front, signal passes through (*bypass*); with a patch cord in front, another signal is tapped or inserted instead.
- **DA (Distribution Amplifier):** takes one input and replicates it across several identical outputs — the model's `DISTRIBUIDOR`.
- **Routing matrix:** connects any of N inputs to any of M outputs, reconfigurable without re-cabling — the model's `ENRUTADOR/MATRIZ`.
- **DSK (Downstream Keyer):** a mixing gate that combines a background signal with an overlay graphic — the typical use case behind the AND/OR logic-rule engine.
- **On-air chain:** the signal path that actually goes to air, as opposed to monitoring, backup or preview routes — what CableDoc's "critical equipment" set exists to mark.
- **GHOST:** an equipment type that exists to complete physical documentation (a terminal strip, a passive pass-through point) but must never be counted in signal-impact analysis — so the graph engine explicitly excludes it from results.

25. Closing

I write this note holding the same bar I'd apply to any system meant to support a real on-air shift: not whether the code is elegant in the abstract, but whether the technician answering a three-a.m. call trusts what the screen tells them. Based on what I could review of the code, the changelog, and the project's own planning documents, that trust is well placed — not because the system is bug-free (the several bugs documented in detail here are enough to disprove that idea), but because every bug found was traced to its exact root cause, fixed with a measurable validation against real data, and written down so the next person to touch that code doesn't have to rediscover the same problem from zero.

Between the first and second editions of this review, that same discipline turned inward: the project spent real, tracked effort refactoring its own largest file down to a 145-line facade, deleting nearly a thousand lines of legacy code it no longer needed rather than dragging it forward, and catching its own extraction bugs with the same rigor it applies to a production signal-impact miscalculation. That is, more than any single feature covered in these pages, the reason it's worth continuing to build on this foundation.

If anything sets CableDoc apart from the large majority of plant-documentation tools I've seen in twenty years of career — many of them glorified spreadsheets, or commercial software that treats cabling as free text — it's that it treats physical topology as what it is: a graph. And once that decision is taken seriously, everything else — impact, risk, diagnosis, contingency planning, workstation sync, a full mobile edition, and now a refactor discipline applied to the tool's own internals — stops being a separate feature and becomes just another query, or another presentation layer, over the same model.

About the author

An engineer with a background in broadcast-plant maintenance (racks, patch bays, transmission chains) who later moved into infrastructure-management software development. Writes on the intersection of domain engineering and software architecture for trade publications. This review was commissioned as an independent technical audit, with no commercial relationship to CableDoc Desktop's development.

Methodology note: this second edition was written directly from the project's full source tree as delivered, including the complete `changelog.txt`*, the SQL schema (*`schema_db.sql`*), the README, the* `plans/` *planning folder, and thirteen user-facing tutorials covering every feature shipped since the first edition. Table, view, and module counts were measured directly against the supplied schema and file list, not estimated; module line counts and the operational figures cited (equipment impacted, patch bays with atypical naming, before/after load times) come from the corresponding changelog entries themselves and were not independently re-verified against a live production database. This edition supersedes and replaces the first (August 2026, Spanish) in full.*